

WEEK 8 ASSIGNMENT REPORT

E-COMMERCE ORDER ANALYTICS SYSTEM

1. INTRODUCTION

The E-Commerce Order Analytics System is an end-to-end data analytics project developed using Python and SQL. The system generates realistic e-commerce datasets, cleans and validates the data using Pandas, stores the cleaned data in an SQLite database, and performs advanced SQL analytics to generate business insights. The project also includes a command-line reporting tool that dynamically generates reports based on user input.

2. OBJECTIVE

The objective of this project is to design and develop an end-to-end e-commerce analytics system that performs data generation, data cleaning, database loading, SQL analytics, customer segmentation, cohort analysis, and command-line reporting while ensuring data quality and robustness.

3. TECHNOLOGIES USED

Technology	Purpose
Python	Dataset generation and scripting
Pandas	Data cleaning and preprocessing
Faker	Synthetic dataset generation
SQLite	Database management
SQL	Business analytics
Tabulate	CLI report formatting
VS Code	Development environment

4. PROJECT WORKFLOW

Submitted By: Jiya Singla(MMDU)



5. IMPLEMENTATION STEPS

Step 1: Dataset Generation

Realistic e-commerce datasets were generated using Python with the **Faker** and **random** libraries. Four CSV files were created:

- customers.csv
- products.csv
- orders.csv
- order_items.csv

Screenshot 1: Step1_Generate_Data_Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)> python scripts/generate_data.py
products.csv saved (500 records)
orders.csv saved (500 records)
order_items.csv saved (700 records)

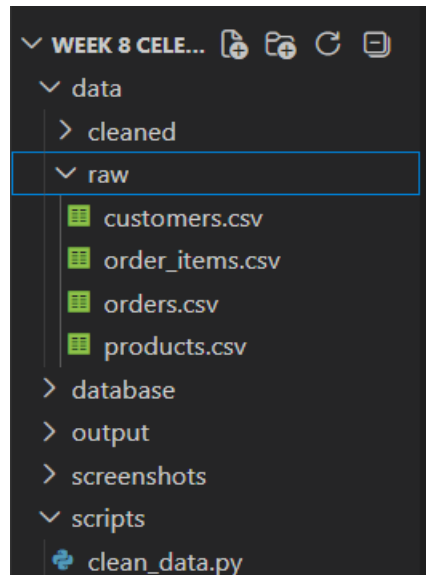
Generation Completed Successfully

Dataset Summary
-----
Customers    : 500
Products     : 500
Orders       : 500
Order Items  : 700
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)>
  
```

Submitted By: Jiya Singla(MMDU)

To simulate real-world data, the datasets include missing values, duplicate records, invalid dates, incorrect email addresses, mismatched IDs, and invalid discount values. The generated files were saved in the **data/raw/** folder.

Screenshot 2: Step1_Raw_CSV_Files



Step 2: Data Cleaning and Validation

The raw datasets were loaded into Pandas DataFrames for cleaning.

The following operations were performed:

- Removed duplicate records
- Handled missing values
- Standardized date formats
- Cleaned product names
- Validated email addresses
- Checked referential integrity between tables
- Identified invalid quantities and discount values

Screenshot 3: Step2_Clean_Data_Output

```
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)> python scripts/clean_data.py

Loading datasets...

customers_clean.csv created
products_clean.csv created
orders_clean.csv created
order_items_clean.csv created
issues_report.txt created

Invalid Email Customer IDs
[42, 138, 207, 237, 390, 391, 442, 493]

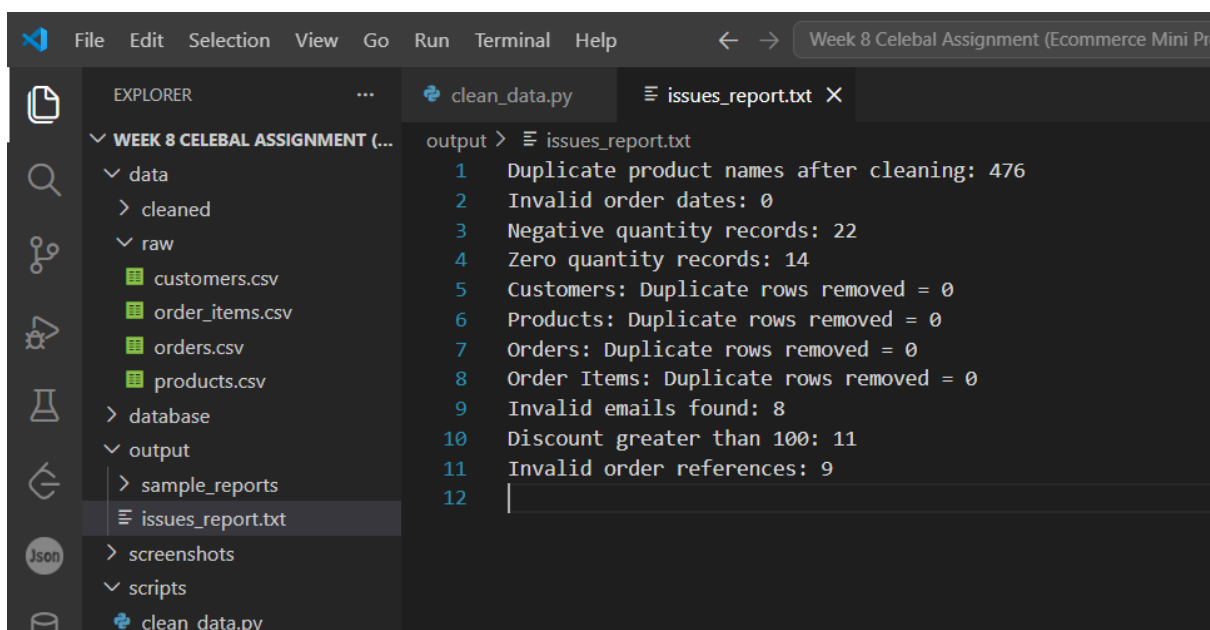
Invalid Order References
  order_item_id  order_id  product_id  quantity  unit_price  discount_percent
12             13       504        351         4      5856.96           16
191            192       507        108         4      5267.97           27
248            249       515        318         3      1505.58            3
265            266       507        451         5      7924.06           14
362            363       520        493         3      3891.21           21
456            457       519        382         4      2207.29           40
462            463       502        395         1      7521.07          101
488            489       510        104         1       863.31            36
664            665       504        392         5      3864.91            6

Cleaning Summary
-----
Customers      : 500
Products       : 500
Orders         : 500
Order Items    : 700
Invalid Emails : 8
Invalid Orders : 9

Data Cleaning Completed Successfully!
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)>
```

An issues_report.txt file was generated to record validation issues.

Screenshot 4: Step2_Issues_Report

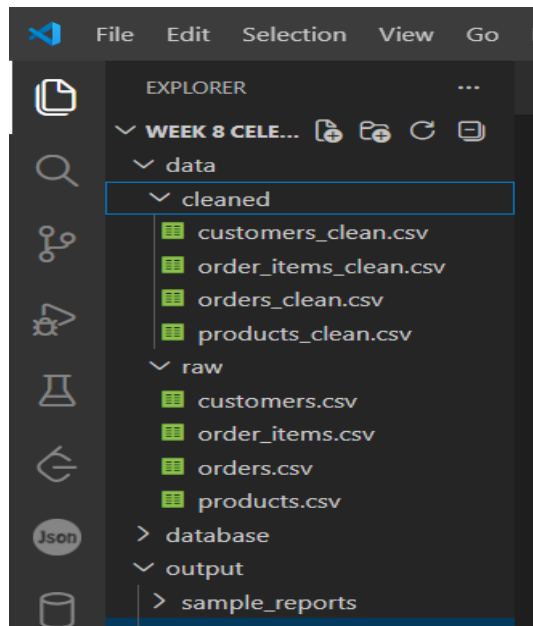


```
output > issues_report.txt
1 Duplicate product names after cleaning: 476
2 Invalid order dates: 0
3 Negative quantity records: 22
4 Zero quantity records: 14
5 Customers: Duplicate rows removed = 0
6 Products: Duplicate rows removed = 0
7 Orders: Duplicate rows removed = 0
8 Order Items: Duplicate rows removed = 0
9 Invalid emails found: 8
10 Discount greater than 100: 11
11 Invalid order references: 9
12
```

The cleaned datasets were stored in the data/cleaned/ folder.

Screenshot 5: Step2_Cleaned_CSV_Files

Submitted By: Jiya Singla(MMDU)



Step 3: Loading Data into SQLite

The cleaned CSV files were imported into an SQLite database.

The database schema was created using Primary Keys, Foreign Keys, and NOT NULL constraints.

Screenshot 6: Step3_SQLite_Database_Tables

customer_id	customer_name	email	registration_date	customer_type	region
1	Brian Yang	johnsonjoshua@example...	2025-11-05	REGULAR	East
2	Jonathan Johnson	garzaanthony@example.o...	2024-05-15	REGULAR	North
3	Abigail Shaffer	lrobinson@example.com	2026-06-30	VIP	North
4	Gabrielle Davis	jpeterson@example.org	2023-12-09	REGULAR	North
5	Kimberly Dudley	melanie94@example.org	2025-02-19	REGULAR	North
6	Heidi Lee	arnoldmaria@example.net	2023-10-28	VIP	West
7	Sharon James	stanleykendra@example...	2025-06-15	VIP	East
8	Timothy Wong	francisco53@example.net	2026-02-12	REGULAR	South
9	William Bowman	amandasanchez@example...	2025-11-23	PREMIUM	East
10	Andrew Stewart	gabrielcameron@examp...	2024-08-30	PREMIUM	North
11	Matthew Foster	clarksherrif@example.net	2026-04-05	REGULAR	East
12	Kimberly Burgess	ithomas@example.org	2025-05-07	VIP	East
13	Peter Callahan Jr.	icox@example.net	2024-03-16	VIP	West
14	Julie King	ycarlson@example.com	2025-11-17	PREMIUM	North
15	Patricia Peterson	frazierdanny@example.n...	2024-10-09	VIP	East
16	Deborah Mason	amoor@example.net	2024-04-14	VIP	North

Relationships between all tables were verified after loading.

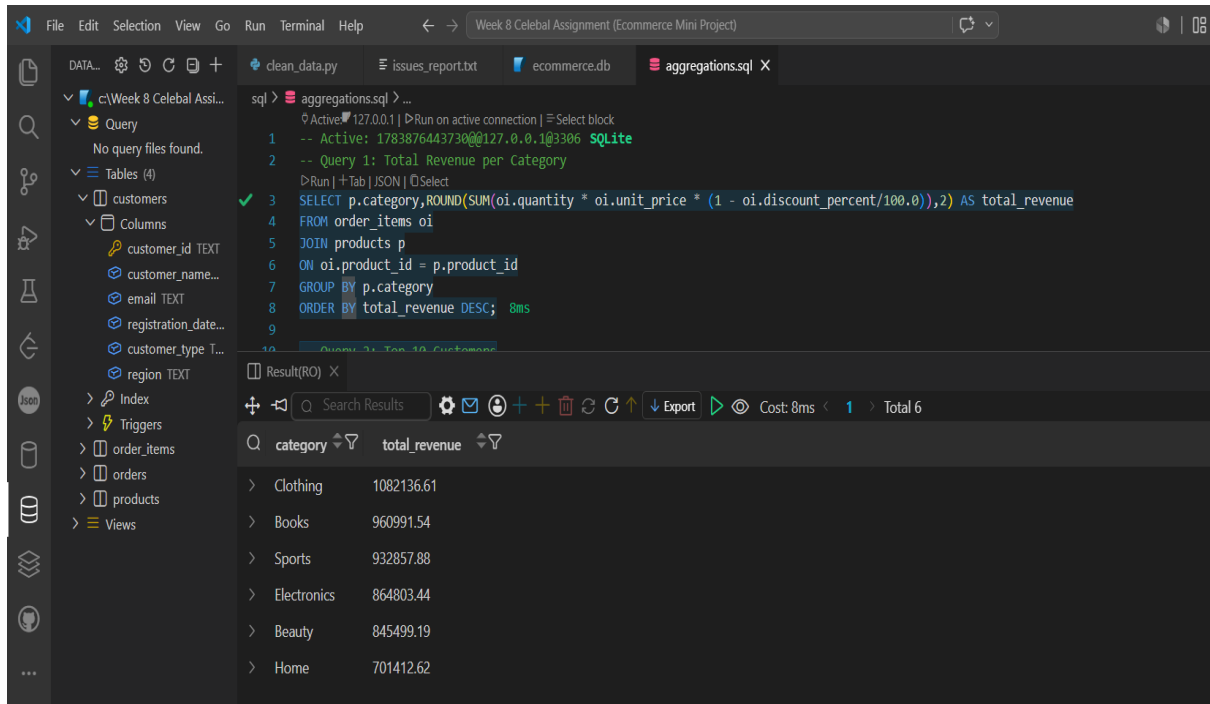
Step 4: SQL Analytics

Submitted By: Jiya Singla(MMDU)

SQL queries were executed to generate business insights.

The SQL analysis includes revenue by category, top customers, monthly order trends, average order value, return rate, customer segmentation, and product performance using joins and aggregation queries.

Screenshot 7: Step4_Revenue_By_Category



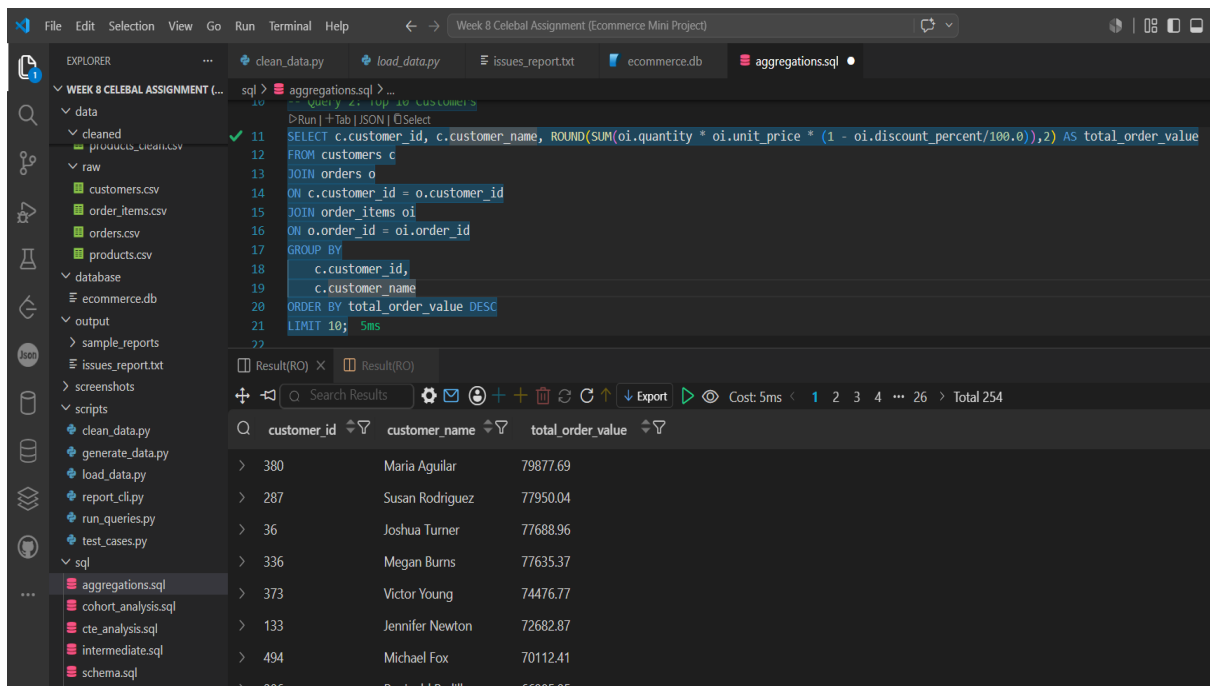
The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```
1 -- Active: 178387644373000127.0.0.1@3306 SQLite
2 -- Query 1: Total Revenue per Category
3 SELECT p.category, ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_percent/100.0)),2) AS total_revenue
4 FROM order_items oi
5 JOIN products p
6 ON oi.product_id = p.product_id
7 GROUP BY p.category
8 ORDER BY total_revenue DESC; 8ms
```

The results pane shows the following data:

category	total_revenue
Clothing	1082136.61
Books	960991.54
Sports	932857.88
Electronics	864803.44
Beauty	845499.19
Home	701412.62

Screenshot 8: Step4_Top_Customer



The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

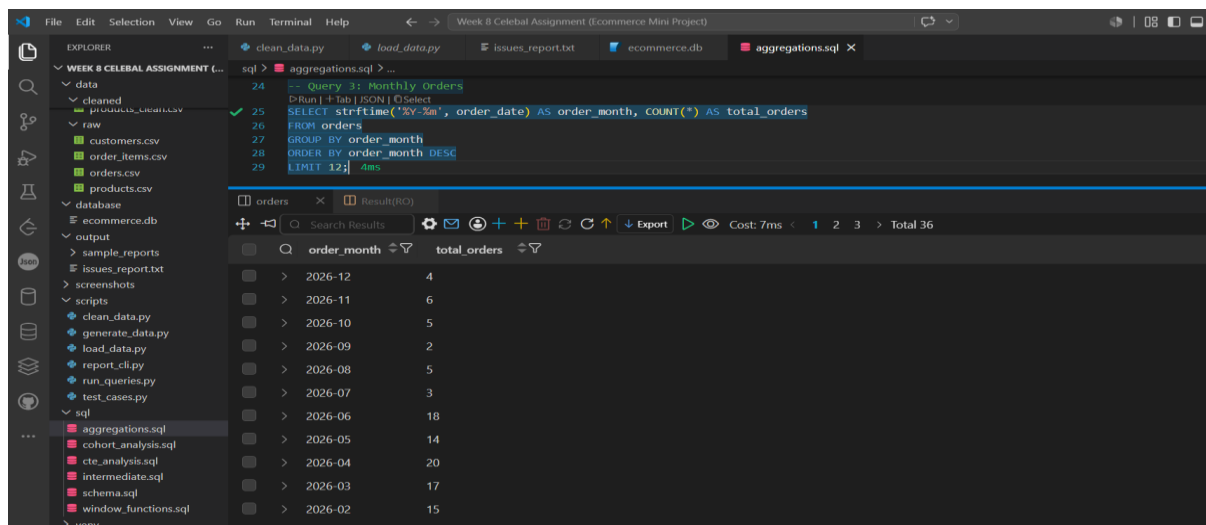
```
11 SELECT c.customer_id, c.customer_name, ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount_percent/100.0)),2) AS total_order_value
12 FROM customers c
13 JOIN orders o
14 ON c.customer_id = o.customer_id
15 JOIN order_items oi
16 ON o.order_id = oi.order_id
17 GROUP BY
18 c.customer_id,
19 c.customer_name
20 ORDER BY total_order_value DESC
21 LIMIT 10; 5ms
```

The results pane shows the following data:

customer_id	customer_name	total_order_value
380	Maria Aguilar	79877.69
287	Susan Rodriguez	77950.04
36	Joshua Turner	77688.96
336	Megan Burns	77635.37
373	Victor Young	74476.77
133	Jennifer Newton	72682.87
494	Michael Fox	70112.41
206	Reginald Padilla	66985.95

Submitted By: Jiya Singla(MMDU)

Screenshot 9: Step4_Monthly_Orders

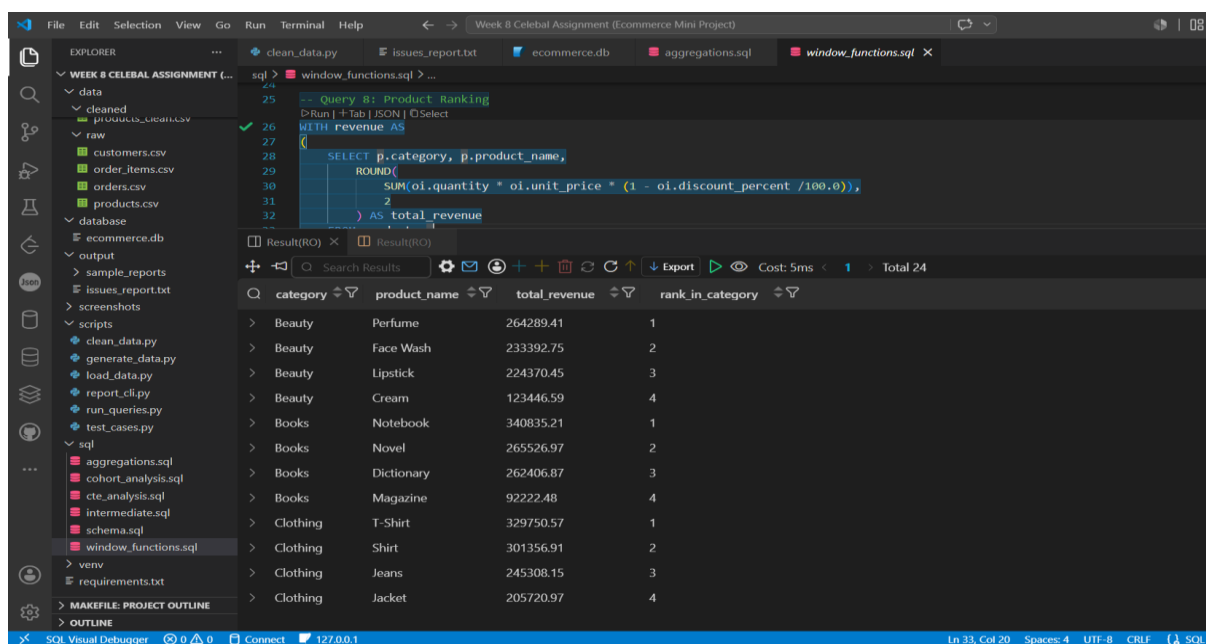


Step 5: Window Functions and CTEs

Advanced SQL analytics were implemented using Window Functions and Common Table Expressions (CTEs).

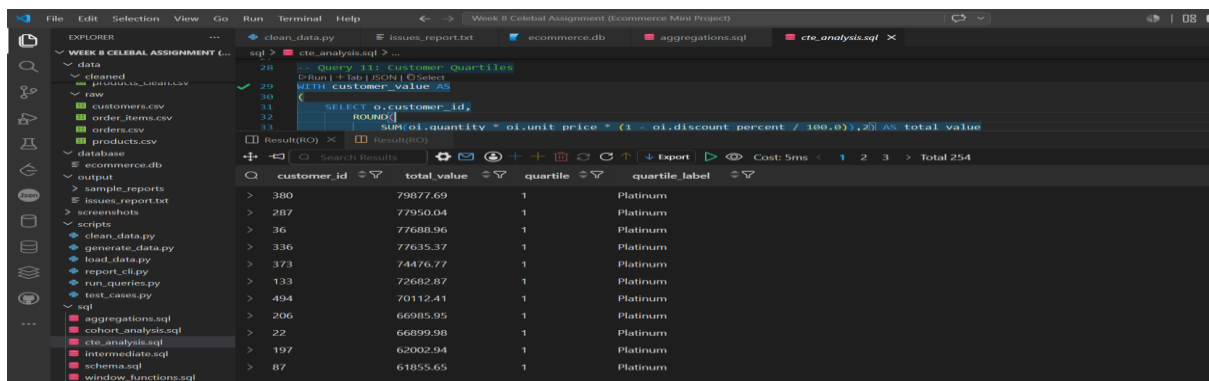
Window functions and CTEs were used to implement running totals, product ranking using `DENSE_RANK()`, customer order gap analysis using `LAG()`, revenue contribution, monthly customer segmentation, customer quartiles using `NTILE()`, and year-over-year revenue comparison.

Screenshot 10: Step5_Product_Ranking



Submitted By: Jiya Singla(MMDU)

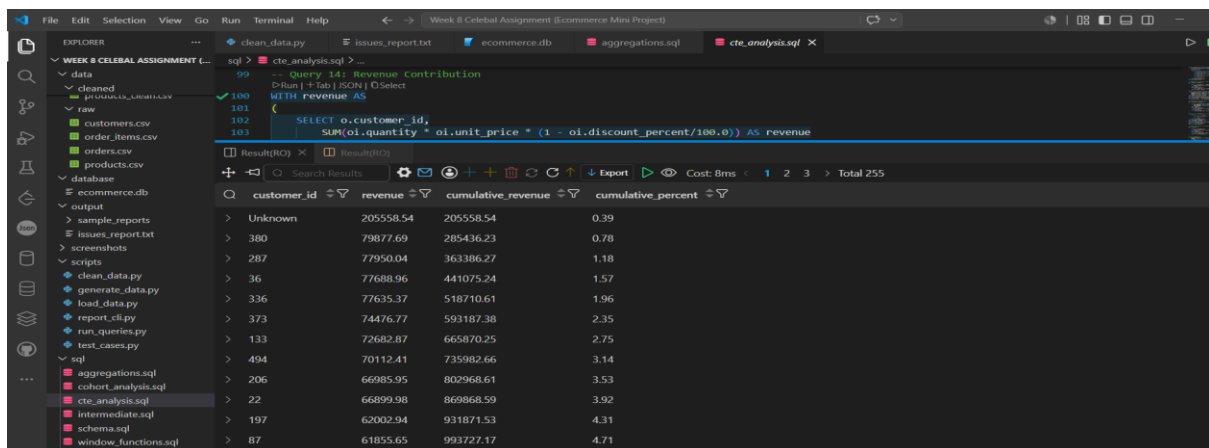
Screenshot 11: Step5_Customer_Quartiles



The screenshot shows a SQL IDE with a query titled "Query 11: Customer Quartiles". The query uses a CTE to calculate the total value for each customer and then assigns them to quartiles based on their total value. The results table shows the following data:

customer_id	total_value	quartile	quartile_label
380	79877.69	1	Platinum
287	77950.04	1	Platinum
36	77688.96	1	Platinum
336	77635.37	1	Platinum
373	74476.77	1	Platinum
133	72682.87	1	Platinum
494	70112.41	1	Platinum
206	66985.95	1	Platinum
22	66899.98	1	Platinum
197	62002.94	1	Platinum
87	61855.65	1	Platinum
52	60927.64	1	Platinum

Screenshot 12: Step5_Revenue_Contribution



The screenshot shows a SQL IDE with a query titled "Query 14: Revenue Contribution". The query calculates the revenue for each customer and then calculates the cumulative revenue and cumulative percent. The results table shows the following data:

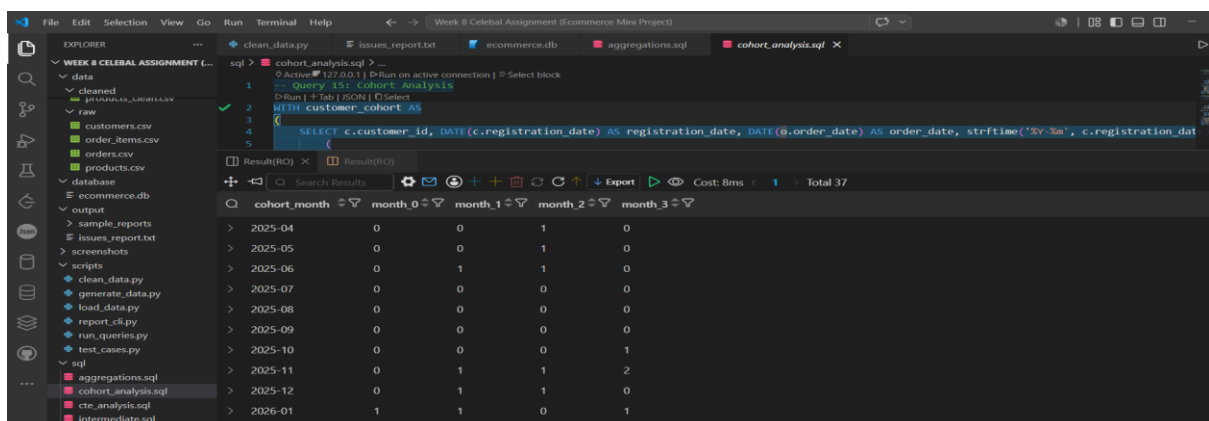
customer_id	revenue	cumulative_revenue	cumulative_percent
Unknown	205558.54	205558.54	0.39
380	79877.69	285436.23	0.78
287	77950.04	363386.27	1.18
36	77688.96	441075.24	1.57
336	77635.37	518710.61	1.96
373	74476.77	593187.38	2.35
133	72682.87	665870.25	2.75
494	70112.41	735982.66	3.14
206	66985.95	802968.61	3.53
22	66899.98	869868.59	3.92
197	62002.94	931871.53	4.31
87	61855.65	993727.17	4.71

Step 6: Cohort and Retention Analysis

Customer cohorts were created based on their registration month.

The analysis includes Cohort Analysis, Monthly Retention Rate, and Repeat Customer Tracking.

Screenshot 13: Step6_Cohort_Analysis

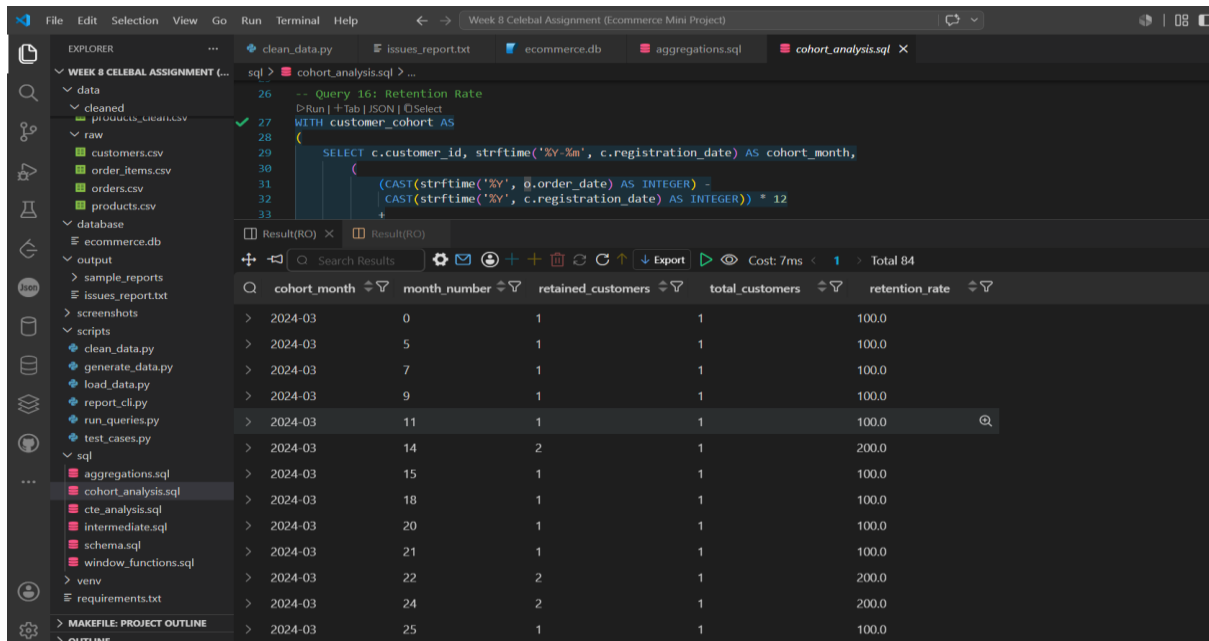


The screenshot shows a SQL IDE with a query titled "Query 15: Cohort Analysis". The query uses a CTE to calculate the cohort month for each customer and then calculates the retention rate for each cohort. The results table shows the following data:

cohort_month	month_0	month_1	month_2	month_3
2025-04	0	0	1	0
2025-05	0	0	1	0
2025-06	0	1	1	0
2025-07	0	0	0	0
2025-08	0	0	0	0
2025-09	0	0	0	0
2025-10	0	0	0	1
2025-11	0	1	1	2
2025-12	0	1	1	0
2026-01	1	1	0	1
2026-02	0	1	1	1

Submitted By: Jiya Singla(MMDU)

Screenshot 14: Step6_Retention_Rate



The screenshot shows a SQL IDE with a query editor and a results table. The query is for 'Query 16: Retention Rate' and calculates the retention rate for each month. The results table has columns: cohort_month, month_number, retained_customers, total_customers, and retention_rate.

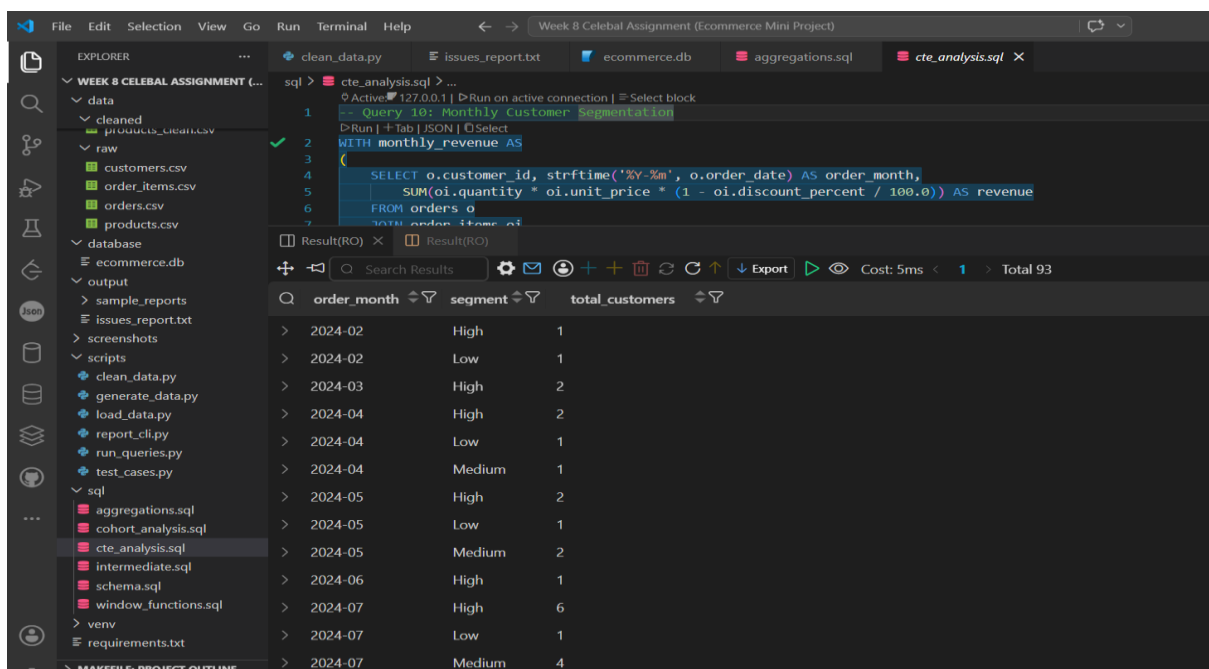
cohort_month	month_number	retained_customers	total_customers	retention_rate
> 2024-03	0	1	1	100.0
> 2024-03	5	1	1	100.0
> 2024-03	7	1	1	100.0
> 2024-03	9	1	1	100.0
> 2024-03	11	1	1	100.0
> 2024-03	14	2	1	200.0
> 2024-03	15	1	1	100.0
> 2024-03	18	1	1	100.0
> 2024-03	20	1	1	100.0
> 2024-03	21	1	1	100.0
> 2024-03	22	2	1	200.0
> 2024-03	24	2	1	200.0
> 2024-03	25	1	1	100.0

Step 7: Customer Segmentation

Customers were segmented based on purchasing behaviour using revenue and purchase frequency.

They were categorized into Platinum, Gold, Silver, and Bronze groups.

Screenshot 15: Step7_Customer_Segmentation

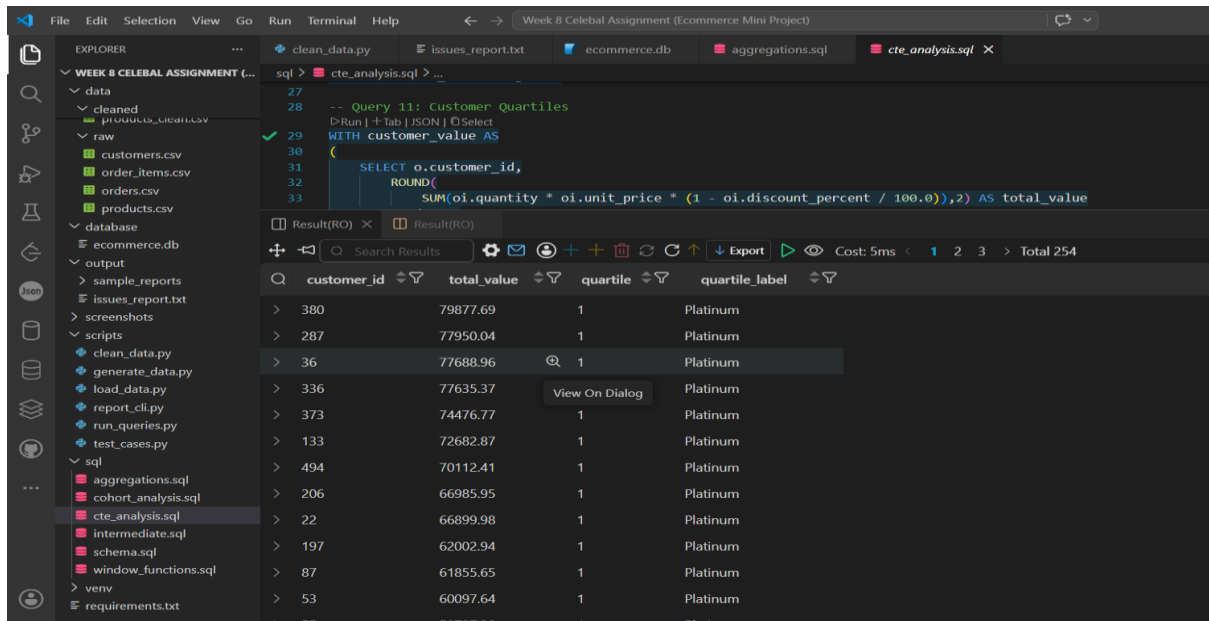


The screenshot shows a SQL IDE with a query editor and a results table. The query is for 'Query 10: Monthly Customer Segmentation' and calculates the segment for each month based on revenue and purchase frequency. The results table has columns: order_month, segment, and total_customers.

order_month	segment	total_customers
> 2024-02	High	1
> 2024-02	Low	1
> 2024-03	High	2
> 2024-04	High	2
> 2024-04	Low	1
> 2024-04	Medium	1
> 2024-05	High	2
> 2024-05	Low	1
> 2024-05	Medium	2
> 2024-06	High	1
> 2024-07	High	6
> 2024-07	Low	1
> 2024-07	Medium	4

Submitted By: Jiya Singla(MMDU)

Screenshot 16: Step7_Quartile_Classification



Step 8: Command-Line Reporting Tool

A Python CLI tool was developed to generate reports directly from the SQLite database.

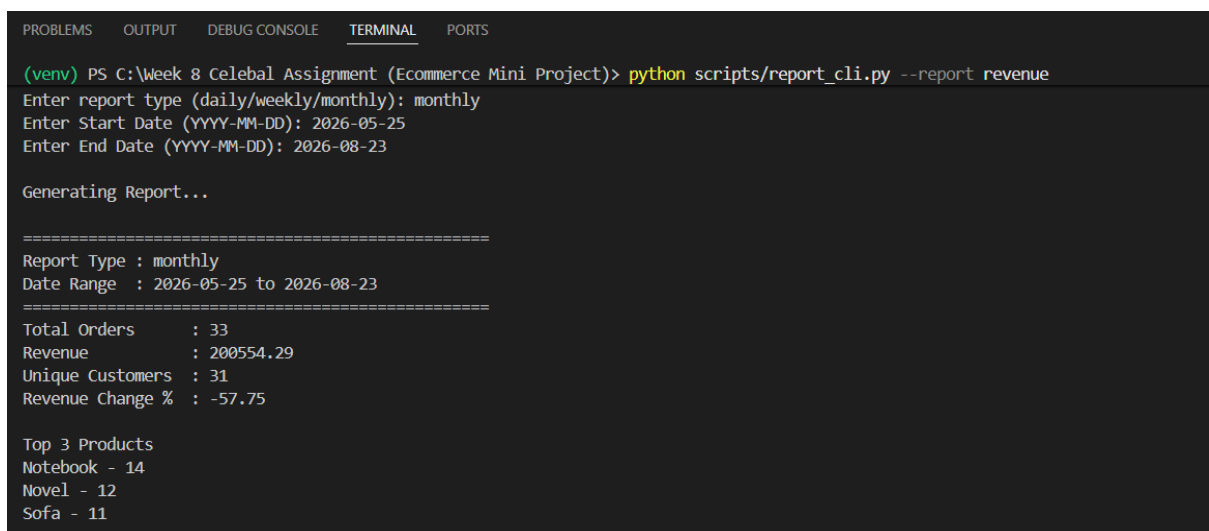
Example commands:

```
python report_cli.py --report revenue
```

```
python report_cli.py --report top_customers
```

```
python report_cli.py --report retention
```

Screenshot 17: Step8_CLI_Execution



Submitted By: Jiya Singla(MMDU)

Screenshot 18: Step8_CLI_Top_Customers_Report

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)> python scripts/report_cli.py --report top_customers
Enter report type (daily/weekly/monthly): weekly
Enter Start Date (YYYY-MM-DD): 2025-04-25
Enter End Date (YYYY-MM-DD): 2026-05-26

Generating Report...

=====
Report Type : weekly
Date Range  : 2025-04-25 to 2026-05-26
=====
Total Orders    : 260
Revenue         : 2804438.57
Unique Customers : 196
Revenue Change % : 34.32

Top 3 Products
T-Shirt - 73
Tennis Racket - 69
Notebook - 62
```

Screenshot 19: Step8_CLI_Retention_Report

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)> python scripts/report_cli.py --report retention
Enter report type (daily/weekly/monthly): daily
Enter Start Date (YYYY-MM-DD): 2026-02-28
Enter End Date (YYYY-MM-DD): 2026-04-25

Generating Report...

=====
Report Type : daily
Date Range  : 2026-02-28 to 2026-04-25
=====
Total Orders    : 36
Revenue         : 334842.29
Unique Customers : 34
Revenue Change % : 4.72

Top 3 Products
Table - 10
Jeans - 9
Cricket Ball - 9
```

Step 9: Edge Case Handling

The system was tested for different scenarios such as missing files, invalid report names, empty datasets, future dates, duplicate records, and database connection failures to ensure reliable execution.

Step 10: Documentation

Submitted By: Jiya Singla(MMDU)

Project documentation was completed by preparing a README.md file containing the project overview, folder structure, setup instructions, execution steps, and sample outputs. All source code, SQL scripts, datasets, and reports were organized according to the required project structure for submission.

6. CONCLUSION

The E-Commerce Order Analytics System successfully implements a complete data analytics pipeline from dataset generation to business reporting. Python was used for data generation, cleaning, and validation, while SQL was used to perform advanced analytics such as joins, aggregations, window functions, customer segmentation, cohort analysis, and retention analysis. The CLI reporting tool enables dynamic report generation, and robust validation ensures reliable data processing. This project demonstrates practical skills in Python, SQL, database management, and analytics.